Remember when I wrote *loosely defined* and *lack of strictness*? Well, POST, PUT, and PATCH are some of the sources of this slipperiness. REST doesn't enforce anything, and if you decide to use POST for both creating and updating, nothing is going to stop you. You have to read an API's documentation to know which methods it uses for what operations, but you'd have had to read the docs to implement it anyway, so that's not that big a deal.

That covers the verbs, but how about the nouns? Like with HTTP, REST uses URLs to determine what object is getting acted upon. Again, this isn't strictly enforced, but there are some best practices that should be followed. Typically, the URL starts with a namespace (e.g. *api*); optionally, includes a grouping (in Django, this might be the app); and then the noun of the object to interact with. REST URLs should not include verbs, as the HTTP protocol specifies the action. This means the same URL can be used for multiple operations. For example, http://example.com/api/song/42/ indicates the song with ID 42. Running GET fetches the song, running PUT expects data in the call to update the song's fields, and running DELETE would remove the song. That's one URL with three different actions.

A common practice is to use plural nouns to retrieve listings and singular nouns and an object ID to fetch specifics. For example, http://example.com/api/songs/ fetches a list of songs in the database, while the aforementioned http://example.com/api/song/42/ fetches a single song. The singular form, without an ID, typically gets used for creation. Following our example, http://example.com/api/song/ called using POST creates a new song.

The POST, PUT, and PATCH methods all expect data in the body of the request that specifies information about the change. This is typically the fields of the object being created or modified. All five HTTP methods usually include data in the body of the response, with everything except DELETE containing the serialized version of the object getting interacted with. DELETE is a special case; some APIs do return the content of the newly deleted object, while others simply return a flag indicating success or failure of the deletion. I tend to do the latter, but you may want the content of the deleted object if you want the user to be able to undo their operation by re-creating it.

REST does not specify the form of all this data getting sent back and forth. The most common format is JSON, while older interfaces still use XML. This is where REST can get a little messy, as there is no mechanism for determining what the body might contain, except making a request. This isn't as troublesome as it might sound. Nobody randomly queries a server with REST. If you're writing code that consumes a REST API, there is probably some documentation somewhere telling you what the API expects. Yep, that's the second reference to reading the docs. Maybe I'm suggesting you'll need to read the docs? Figure 11.2 summarizes the use of HTTP methods with JSON data to perform CRUD operations through a REST API.

This figure has all the operations needed to build a song API. Consider a mobile application where a user can input and modify a database of their favorite songs. The application's home screen lists all the songs in their database, by calling GET on /api/songs/. When the user clicks on the song "Fire" by Ohio Players, they see the details,